

2026编译技术实验文法定义及相关说明

一、概要

SysY 语言是编译技术实验所完成的编译器的源语言，是 C 语言的一个子集。一个 SysY 语言源程序文件中有且仅有一个名为 `main` 的主函数定义，除此之外包含若干全局变量声明、常量声明和其他函数定义。SysY 语言支持 32 位有符号数 `int` 类型、8 位无符号数 `char` 类型及其的一维数组类型；`const` 修饰符用于声明常量。

SysY 语言本身没有提供输入/输出(I/O)的语言构造，I/O 是以运行时库方式提供，库函数可以在 SysY 程序中的函数内调用。

- 函数：**函数可以带参数也可以不带参数，参数的类型可以是 `int`、`char` 或其一维数组类型；函数可以返回 `int`、`char` 类型的值，或者不返回值（即声明为 `void` 类型）。当参数为 `int` 或 `char` 时，按值传递；而参数为数组类型时，实际传递的是数组的起始地址。函数体由若干变量声明和语句组成。
- 变量/常量声明：**可以在一个变量/常量声明语句中声明多个变量或常量，声明时可以带初始化表达式。所有变量/常量要求先定义再使用。在函数外声明的为全局变量/常量，在函数内声明的为局部变量/常量。
- 语句：**语句包括赋值语句、表达式语句(表达式可以为空)、语句块、`if` 语句、`while` 语句、`switch` 语句、`break` 语句、`continue` 语句、`return` 语句。语句块中可以包含若干变量声明和语句。
- 表达式：**支持基本的算术运算（+、-、*、/、%）、关系运算（==、!=、<、>、<=、>=）和逻辑运算（!、&&、||），非 0 表示真、0 表示假，而关系运算或逻辑运算的结果用 1 表示真、0 表示假。**算符的优先级和结合性以及计算规则(含逻辑运算的“短路计算”)与 C 语言一致。**

二、运行时库

源程序可通过 `get_int`、`get_char`、`get_string` 读取输入，通过 `put_int`、`put_char`、`put_string`、`printf` 打印输出，相关函数的原型及作用如下：

函数名称	函数原型	函数作用
<code>get_int</code>	<code>int get_int();</code>	从标准输入读取一行，去掉首尾的空白字符，将剩余部分按十进制整数的规则解析，保证不出现无法解析为十进制数字的情况，保证不出现到达文件末尾（EOF）后仍调用的情况。该函数的行为与 MARS 模拟器 5 号系统调用（ <code>read integer</code> ）相同。
<code>get_char</code>	<code>char get_char();</code>	从标准输入读取一个字符，并返回其 ASCII 码，保证不出现到达文件末尾（EOF）后仍调用的情况。该函数的行为与 MARS 模拟器 12 号系统调用（ <code>read character</code> ）相同。
<code>get_string</code>	<code>void get_string(char* buffer, int max_len);</code>	从标准输入不超过 <code>max_len - 1</code> 个字符，并将读取到的字符存储到 <code>buffer</code> 指向的缓冲区中。当遇到 EOF 或者换行符时，则停止读取。保证不出现到达文件末尾（EOF）后仍调用的情况。若读取到换行符，则其将存入缓冲区中。若 <code>max_len > 0</code> ，则保证函数返回后， <code>buffer</code> 中含有合法的以空字符结尾的字符串；若 <code>max_len <= 0</code> ，则不执行任何操作，也不修改 <code>buffer</code> 。该函数的行为与MARS模拟器 8 号系统调用（ <code>read string</code> ）相同。

函数名称	函数原型	函数作用
put_int	void put_int(int a);	向标准输出以有符号十进制格式打印一个整数，不自动换行。该函数的行为与 MARS 模拟器 1 号系统调用（print integer）相同。
put_char	void put_char(char a);	向标准输出打印一个字符，不自动换行。该函数的行为与 MARS 模拟器 11 号系统调用（print character）相同。
put_string	void put_str(char *str);	向标准输出打印一个字符串，不自动换行。该函数的行为与 MARS 模拟器 4 号系统调用（print string）相同。

printf 的使用方法为 'printf' '(' <StringConst> '{, <Exp>} ') '，其中 **StringConst** 为字符串常量终结符，其规范参考第三章文法及测试程序覆盖要求的第三小节第五部分字符串常量。

如果希望在 C 语言中测试测试程序，请从希冀平台“课程信息”--“教学资料”中下载 libsysy.zip，引入其中的 libsysy.h 头文件。例如：

```
#include "libsysy.h"

int main() {
    int value = get_int();

    printf("value = %d\n", value);

    return 0;
}
```

编译测试程序时，请将其同 libsysy.c 一同编译，例如，将上述测试程序保存到 test.c 中，使用 gcc 编译器：

```
gcc test.c libsysy.c -o test
```

三、文法及测试程序覆盖要求

1. 覆盖要求

测试程序是为了测试编译器而编写的符合文法规则的 SysY 语言程序，在实验的“文法解读作业”中需要同学们编写测试程序。测试程序需覆盖文法中所有的语法规则（即每一条推导规则的每一个候选项都要被覆盖），并满足文法的语义约束（换言之，测试程序应该是正确的）。在下一节中，文法正文中以注释形式给出需要覆盖的情况。

2. 文法

SysY 语言的文法采用扩展的 Backus 范式（EBNF，Extended Backus-Naur Form）表示，其中：

- 符号 [...] 表示方括号内包含的为可选项

- 符号`{...}`表示花括号内包含的为可重复 0 次或多次的项
- 终结符或者是由单引号括起的串，或者是 `Ident`、`IntConst`、`StringConst` 这样的记号
- 所有类似 `'main'` 这样的用单引号括起的字符串都是保留的关键字

SysY 语言的文法表示如下，其中 `CompUnit` 为开始符号：

重要：建议同时对照文法第三部分的语义约束。

编译单元 `CompUnit` $\rightarrow$ `{Decl} {FuncDef} MainFuncDef` // 1.是否存在Decl 2.是否存在FuncDef

声明 `Decl` $\rightarrow$ `ConstDecl | VarDecl` // 覆盖两种声明

常量声明 `ConstDecl` $\rightarrow$ `'const' BType ConstDef { ',' ConstDef } ';'` // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次，即，至少出现3次ConstDef)

基本类型 `BType` $\rightarrow$ `'int' | 'char'` // 覆盖两种基本类型

常量定义 `ConstDef` $\rightarrow$ `Ident [ '[' ConstExp ']' ] '=' ConstInitVal` // 包含普通变量、一维数组两种情况

常量初值 `ConstInitVal` $\rightarrow$ `ConstExp | '{' [ ConstExp { ',' ConstExp } ] '}' | StringConst` // 1.常表达式初值 2.一维数组初值 3. 字符串初值

变量声明 `VarDecl` $\rightarrow$ `[ 'static' ] BType VarDef { ',' VarDef } ';'` // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次，即，至少出现3次VarDef) 3. `static` 存在 4. `static` 不存在

变量定义 `VarDef` $\rightarrow$ `Ident [ '[' ConstExp ']' ] | Ident [ '[' ConstExp ']' ] '=' InitVal` // 包含普通常量、一维数组定义

变量初值 `InitVal` $\rightarrow$ `Exp | '{' [ Exp { ',' Exp } ] '}' | StringConst` // 1.表达式初值 2.一维数组初值 3. 字符串初值

函数定义 `FuncDef` $\rightarrow$ `FuncType Ident '(' [FuncFParams] ')' Block` // 1.无形参 2.有形参

主函数定义 `MainFuncDef` $\rightarrow$ `'int' 'main' '(' ')'` `Block` // 存在main函数

函数类型 `FuncType` $\rightarrow$ `'void' | 'int' | 'char'` // 覆盖三种类型的函数

函数形参表 `FuncFParams` $\rightarrow$ `FuncFParam { ',' FuncFParam }` // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次，即，至少出现3次FuncFParam)

函数形参 `FuncFParam` $\rightarrow$ `BType Ident '[' ']'` // 1.普通变量 2.一维数组变量

语句块 `Block` $\rightarrow$ `'{' { BlockItem } '}'` // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次)

语句块项 `BlockItem` $\rightarrow$ `Decl | Stmt` // 覆盖两种语句块项

语句 Stmt $\rightarrow$ LVal '=' Exp ';' // 每种类型的语句都要覆盖
 | [Exp] ';' // 有无Exp两种情况; printf函数调用
 | Block
 | 'if' '(' Cond ')' Stmt ['else' Stmt] // 1.有else 2.无else
 | 'while' '(' Cond ')' Stmt // while循环语句
 | 'switch' '(' Exp ')' '{' { CaseStmt } '}' // switch语句, 需覆盖不同case情况
 | 'break' ';' ;
 | 'continue' ';' ;
 | 'return' [Exp] ';' // 1.有Exp 2.无Exp
 | 'printf' '(' StringConst { ',' Exp } ')' ';' // 1.有Exp 2.无Exp

Case语句 CaseStmt $\rightarrow$ 'case' Number ':' { Stmt } | 'default' ':' { Stmt } //
 1.case语句 2.default语句

表达式 Exp $\rightarrow$ AddExp // 存在即可

条件表达式 Cond $\rightarrow$ LOrExp // 存在即可

左值表达式 LVal $\rightarrow$ Ident '[' Exp ']' // 1.普通变量、常量 2.一维数组

基本表达式 PrimaryExp $\rightarrow$ '(' Exp ')' | LVal | Number // 1. 嵌套括号表达式 2. 左值表达式 3. 数值

数值 Number $\rightarrow$ IntConst | CharConst // 存在即可, 覆盖整型常量和字符常量

一元表达式 UnaryExp $\rightarrow$ PrimaryExp | Ident '(' [FuncRParams] ')' | UnaryOp
 UnaryExp | '(' BType ')' UnaryExp // 1.PrimaryExp 2.函数调用且无 FuncRParams
 3.函数调用且存在 FuncRParams 4.UnaryOp UnaryExp 5.显式类型转换

单目运算符 UnaryOp $\rightarrow$ '+' | '-' | '!' 注: '!'仅出现在条件表达式中 // 三种均需覆盖

函数实参表达式 FuncRParams $\rightarrow$ Exp { ',' Exp } // 1.花括号内重复0次 2.花括号内重复多次(花括号内重复 ≥ 2 次, 即, 至少出现3次Exp) 3.Exp需要覆盖数组传参和数组元素传参

乘除模表达式 MulExp $\rightarrow$ UnaryExp | MulExp ('*' | '/' | '%') UnaryExp //
 1.UnaryExp 2.* 3./ 4.% 均需覆盖

加减表达式 AddExp $\rightarrow$ MulExp | AddExp ('+' | '-') MulExp // 1.MulExp 2.+ 需覆盖
 3.- 需覆盖

关系表达式 RelExp $\rightarrow$ AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp //
 1.AddExp 2.< 3.> 4.<= 5.>= 均需覆盖

相等性表达式 EqExp $\rightarrow$ RelExp | EqExp ('==' | '!=') RelExp // 1.RelExp 2.==
 3.!= 均需覆盖

逻辑与表达式 LAndExp $\rightarrow$ EqExp | LAndExp '&&' EqExp // 1.EqExp 2.&& 均需覆盖

逻辑或表达式 LOrExp $\rightarrow$ LAndExp | LOrExp '||' LAndExp // 1.LAndExp 2.|| 均需覆盖

常量表达式 ConstExp $\rightarrow$ AddExp 注: 使用的 Ident 必须是常量 // 存在即可

3. 补充说明

(1) 标识符 Ident

SysY 语言中标识符 **Ident** 的规范如下 (**identifier**) :

```
identifier → identifier-nondigit
           | identifier identifier-nondigit
           | identifier digit
```

其中, **identifier-nondigit** 为下划线或大小写字母, **digit** 为0到9的数字。

注: 请参考 ISO/IEC 9899 <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf> 第 51 页关于标识符的定义同名标识符的约定:

- 全局变量（常量）和局部变量（常量）的作用域可以重叠，重叠部分局部变量（常量）优先；
- 同名局部变量的作用域不能重叠，即同一个 **Block** 内不能有同名的标识符（常量/变量）；
- 在不同的作用域下，SysY 语言中变量（常量）名可以与函数名相同；
- 保留关键字不能作为标识符。

```
// 合法样例
int main() {
    int a = 0;
    {
        int a = 1;
        printf("%d", a); // 输出 1
    }
    return 0;
}
```

```
// 非法样例
int main() {
    int a = 0;
    {
        int a = 1;
        int a = 2; // 非法!
    }
    return 0;
}
```

(2) 注释

SysY 语言中注释的规范与 C 语言一致，如下：

- 单行注释：以序列 **//** 开始，直到换行符结束，不包括换行符。

- 多行注释：以序列 `/*` 开始，直到第一次出现 `*/` 时结束，包括结束处 `*/`。

注：请参考 ISO/IEC 9899 <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf> 第 66 页关于注释的定义。

(3) 数值常量

SysY 语言中数值常量可以是整型数 **IntConst**，其规范如下（对应 `integer-const`）：

```
整型常量 integer-const → decimal-const | 0
decimal-const → nonzero-digit | decimal-const digit
nonzero-digit 为1至9的数字。
```

注：请参考 ISO/IEC 9899 <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf> 第 54 页关于整型常量的定义，在此基础上忽略所有后缀。

(4) 字符常量

SysY 语言中字符常量可以是 **CharConst**，其规范如下：

```
字符常量 CharConst → '\'' <CharElement> '\''
<CharElement> → <NormalChar> | <EscapeChar>
<NormalChar> → 除 '\'', '\\\' 外的可见ASCII字符，即：ASCII码范围[0x20, 0x7E]内的字符，但不包含0x27和0x5C
<EscapeChar> → '\\n' | '\\t' | '\\r' | '\\0' | '\\\\' | '\\\\'
```

注：字符常量用于表示单个字符，其值为对应字符的 ASCII 码值。字符常量可以赋给 `char` 类型变量，但不能直接赋给 `int` 类型变量，除非使用显式类型转换。

(5) 字符串常量

SysY 语言中 **StringConst** 的定义如下：

```
<FormatChar> → %d | %c | %s
<NormalChar> → 十进制编码为32, 33, 40-126的ASCII字符，'\''（编码92）出现当且仅当为'\n'
<Char> → <FormatChar> | <NormalChar>
<StringConst> → '{<Char>}'
```

字符串中仅可能会出现一种转义字符 `'\n'`，用以标注此处换行，其他转义情况无需考虑。

注：printf 函数中，%d 用于输出 int 类型值，%c 用于输出 char 类型值，%s 用于输出 string(char[]) 类型值。

4. 难度分级

为了更好的帮助同学们完成编译器的设计，课程组将题目难度做了区分。难度等级分为三级：A、B、C，难度依次递减：

- C：最简单的等级，重点考察编译器的基础设计，不包括任何与数组有关的部分，if 与 while 中的条件表达式也只有一个表达式（即不出现 && 和 || ）不考察短路求值的规则，不考察 char 类型与 switch 语句。
- B：在 C 级的基础上，新增数组，包括数组定义，数组元素的使用、数组传参等，新增 char 类型与 switch 语句。
- A：在 B 级的基础上，新增复杂条件的运算和判断，要遵守短路求值的规则。
 - 短路求值样例：

```
int global_var = 0;
int func() {
    global_var = global_var + 1;
    return 1;
}
int main() {
    if (0 && func()){
        ;
    }
    printf("%d",global_var); // 输出 0
    if (1 || func()) {
        ;
    }
    printf("%d",global_var); // 输出 0
    return 0;
}
```

四、语义约束

符合上述文法的程序集合是合法的 SysY 语言程序集合的超集。下面进一步给出 SysY 语言的语义约束。

对于本节明确表述的“未定义”的行为，要求编写文法解读程序时不得出现，保证所有测试程序不出现。

运行时库

1. 为了降低开发难度，保证所有测试程序中 get_char、get_int、get_string、put_int、put_char、put_str 与 printf 只会作为运行时库调用出现，不会作为自定义的变量/函数标识符。
2. printf 函数默认不换行。
3. 与 C 语言中的 printf 类似，输出语句中，格式字符将被替换为对应标识符，普通字符原样输出。
4. 若选择 LLVM IR 作为目标代码，则只需在 LLVM IR 中对 get_int、get_char、get_string、put_int、put_char、put_str 进行声明，将产生的 LLVM IR 与 libsysy.zip 中的 libsysy.ll

进行链接即可运行，注意 LLVM IR 中不能直接使用 `printf`，而需要将其转换为 `put_*` 调用。若选择 MIPS 汇编作为目标代码，则需要使用恰当的系统调用实现上述函数的功能。

static 关键字

1. `static` 关键字用于修饰**静态局部变量**，不会在全局变量定义中出现。
2. 用 `static` 修饰的变量声明，即使在声明时未赋初值，编译器也会把它初始化为0。
3. 用 `static` 修饰的变量声明若有初始值，其初始值保证在编译期内可求出。
4. `static` 关键字指定的变量只初始化一次，并在之后调用该函数时保留其状态，离开函数时不会被销毁。

类型约束

1. 为降低教学复杂度，SysY 语言中 `int` 和 `char` 是两种不同的基本类型，不允许直接进行混合运算，但允许进行显式的类型转换（详见下），转换后类型一致则可以混合运算。
2. 当表达式中出现多类型（包括 `int/char/int[]/char[]/void` 任意类型）混合运算（且未正确使用显式类型转换）时，应报类型不匹配错误。
3. `char` 类型变量可以单独使用，也可以进行 `char` 类型之间的运算，运算结果仍为 `char` 类型。
4. `int` 类型变量可以单独使用，也可以进行 `int` 类型之间的运算，运算结果仍为 `int` 类型。
5. 在赋值语句中，左值和右值的类型必须一致，即 `int` 变量只能赋 `int` 值，`char` 变量只能赋 `char` 值。
6. 数组下标、数组长度只能使用 `int` 类型的表达式，不能使用 `char` 类型的表达式。
7. 函数参数传递时，实参类型必须与形参类型完全一致。
8. 函数返回值类型必须与函数声明的返回类型一致。
9. `char` 类型定义为 **unsigned char**（无符号8位整数），取值范围为 0 ~ 255。此定义符合 ISO/IEC 9899 C 语言标准中 **unsigned char** 的规范。
10. `char` 类型运算发生溢出时，采用 **模256回绕 (wrap-around)** 方式处理。例如：255 + 1 的结果为 0，0 - 1 的结果为 255。此行为符合 C 语言标准中 **unsigned** 整数的溢出处理规则。
11. `char` 类型变量的值范围为 0 ~ 255，其中 0 ~ 127 对应标准 ASCII 字符，可通过字符常量赋值；128 ~ 255 只能通过运算溢出或直接赋数值产生。`printf` 使用 `%c` 输出时，编译器只需确保正确传递 `char` 值给 `printf` 函数，输出结果由运行环境决定。测试用例应避免使用 `%c` 输出 128 ~ 255 的值。
12. 当字符串常量用于初始化 `char` 或 `char[]` 时，应将字符串的字节序列（按转义处理后得到的字节）逐字节复制到数组中；若数组长度大于字符串长度，则剩余元素置 0；若数组长度 `N` 小于字符串长度，则只复制字符串的前 `N` 个字节，不要额外添加 `\0`。
13. 显式类型转换语法 (**BType**) **UnaryExp** 用于将基本类型表达式进行显式转换。被转换的表达式 **UnaryExp** 的类型必须是基本类型 `int` 或 `char`。转换后表达式的类型为目标基本类型 **BType**。显式类型转换可以解决由于类型不匹配导致的混合运算、赋值、传参或返回等错误。

注：打印时，`%d` 用于输出 `int`，`%c` 用于输出 `char`。当向运行时库传递 `char` 时，编译器应确保按目标平台 ABI 正确传递 8 位无符号值（通常会经过整型提升或按 ABI 要求扩展到机器整型）。测试用例中尽量使用 ASCII 范围 (0..127) 的字符常量和字符串，以避免平台编码差异影响结果。

注：此约束旨在在语义阶段时就建构类型系统，避免在中间代码生成时再对前端部分进行调整。

编译单元 CompUnit

编译单元 `CompUnit` $\rightarrow$ `{Decl} {FuncDef} MainFuncDef`

声明 $\text{Decl} \rightarrow \text{ConstDecl} \mid \text{VarDecl}$

1. **CompUnit** 的顶层变量/常量声明语句（对应 **Decl**）、函数定义（对应 **FuncDef**）都不可以重复定义同名标识符（**Ident**），即便标识符的类型不同也不允许。
2. **CompUnit** 的变量/常量/函数声明的作用域从该声明处开始到文件结尾。

常量定义 ConstDef

常量定义 $\text{ConstDef} \rightarrow \text{Ident} \text{ ['[' ConstExp ']'] '=' ConstInitVal}$

1. **ConstDef** 用于定义符号常量。**ConstDef** 中的 **Ident** 为常量的标识符，在 **Ident** 后、**=** 之前是可选的一维数组以及一维数组长度的定义部分，在 **=** 之后是初始值。**常量必须有对应的初始值。**
2. **ConstDef** 的一维数组以及一维数组长度的定义部分不存在时，表示定义单个变量。
3. **ConstDef** 的一维数组以及一维数组长度的定义部分存在时，表示定义数组。其语义和 C 语言一致，每维的下界从 0 编号。
4. **ConstDef** 中表示各维长度的 **ConstExp** 都必须是整型常量表达式。

变量定义 VarDef

变量定义 $\text{VarDef} \rightarrow \text{Ident} \text{ ['[' ConstExp ']']} \mid \text{Ident} \text{ ['[' ConstExp ']'] '=' InitVal}$

1. 变量的定义与常量基本一致，但是变量可以没有初始值部分，此时其运行时实际初值未定义。

初值 ConstInitVal / InitVal

常量初值 $\text{ConstInitVal} \rightarrow \text{ConstExp} \mid \text{'{' [ConstExp { ',' ConstExp }] '}' \mid \text{StringConst}$

变量初值 $\text{InitVal} \rightarrow \text{Exp} \mid \text{'{' [Exp { ',' Exp }] '}' \mid \text{StringConst}$

1. 任何常量的初值表达式必须是常量表达式 **ConstExp**。
2. **常量必须有初始值**，常量数组不需要每一个元素都有初始值，但是未赋值的部分编译器需要将其置0。
3. 全局变量的初值表达式也必须是常量表达式 **ConstExp**。但局部变量的初值表达式是 **Exp**，可以使用已经声明的变量。
4. 对于单个的常量或变量，初始值也必须是单个的表达式。（表达式包括单个数字、单个变量、单个常量等情况）
5. 对于全局变量，如果没有给出初始值，那么应该全部置 0，局部变量不需要。

函数形参 FuncFParam 与实参 FuncRParams

函数形参表 $\text{FuncFParams} \rightarrow \text{FuncFParam} \{ ', ' \text{FuncFParam} \}$
 函数实参表 $\text{FuncRParams} \rightarrow \text{Exp} \{ ', ' \text{Exp} \}$

1. **FuncFParam** 定义一个函数的一个形式参数。当 **Ident** 后面的可选部分存在时，表示数组定义。
2. 函数实参的语法是 **Exp**。对于普通变量，遵循按值传递；对于数组类型的参数，则形参接收的是实参数组的地址，并通过地址间接访问实参数组中的元素。
3. 函数调用时要保证实参类型和形参类型一致，具体请看下面例子。

```
void f1(int x){
    return ;
}
void f2(int x[]){
    return ;
}

int main(){
    int x = 10;
    int t[5] = {1, 2, 3, 4, 5};
    f1(x); // 合法
    f1(t[0]); // 合法
    f1(t); // 不合法
    f2(t); // 合法
}
```

函数定义 FuncDef

函数定义 $\text{FuncDef} \rightarrow \text{FuncType} \text{ Ident } ' (' [\text{FuncFParams}] ') ' \text{ Block}$

1. **FuncDef** 表示函数定义。其中的 **FuncType** 指明返回类型。当返回类型为 **int** 时，函数内所有分支都应当含有带有 **Exp** 的 **return** 语句。不含有 **return** 语句的分支的返回值未定义。当返回值类型为 **void** 时，函数内只能出现不带返回值的 **return** 语句。
2. **FuncFParams** 的语义如前文。

语句块 Block

1. **Block** 表示语句块。语句块会创建作用域，语句块内声明的常量和变量的生存期在该语句块内。
2. 语句块内可以再次定义与语句块外同名的变量或常量 (通过 **Decl** 语句)，其作用域从定义处开始到该语句块尾结束，它隐藏语句块外的同名变量或常量。
3. 为了降低开发难度，保证所有测试程序中有返回值的函数 **Block** 的最后一句一定会显式的给出 **return** 语句，否则就当作“无返回语句”的错误处理。同时，同学们编写上传的样例程序时，也需要保证这一点。
4. **main** 函数的返回值只能为常数0。

语句 Stmt

1. **Stmt** 中的 **if** 类型语句遵循就近匹配，即，在没有使用大括号明确界定代码块范围的情况下，一个 **else** 语句总是与它上方距离它最近的、且尚未配对的 **if** 语句结合（配对）。
2. 单个 **Exp** 可以作为 **Stmt**。这个 **Exp** 会被求值（这可能产生副作用），但是所求的值会被丢弃。
3. **while** 语句中，**Cond** 为循环条件，**Stmt** 为循环体。当 **Cond** 为真时，执行 **Stmt**，否则跳出循环。
4. **switch** 语句中，**Exp** 为 **switch** 表达式，其值用于与各个 **case** 的 **Number** 进行匹配。
5. **switch** 语句中的每个 **case** 后必须有 **Number** 作为匹配值。**Number** 的类型必须与 **switch** 表达式 **Exp** 的类型一致：若 **Exp** 为 **int** 类型，**Number** 必须为 **int** 常量 (**IntConst**)；若 **Exp** 为 **char** 类型，**Number** 必须为 **char** 常量 (**CharConst**)。
6. **switch** 语句中最多可以有一个 **default** 语句，当所有 **case** 都不匹配时执行 **default** 后的语句。
7. **break** 语句可以出现在循环语句 (**while**) 或 **switch** 语句中，用于跳出循环或 **switch** 结构。
8. **continue** 语句只能出现在循环语句 (**while**) 中，用于跳过当前迭代，继续下一次循环。
9. **switch** 语句中，**break** 用于跳出 **switch** 结构，若某个 **case** 后没有 **break**，则继续执行下一个 **case** 的语句。

左值 LVal

1. **LVal** 表示具有左值的表达式，可以为变量或者某个数组元素。
2. 当 **LVal** 表示数组时，方括号个数必须和数组变量的维数相同（即定位到元素）。
3. 当 **LVal** 表示单个变量时，不能出现后面的方括号。

Exp 与 Cond

1. **Exp** 在 **SysY** 中代表算术表达式；**Cond** 代表条件表达式，故它定义为 **LORExp**。前者的单目运算符中不出现 **!**，后者可以出现。
2. **LVal** 必须是当前作用域内、该 **Exp** 语句之前有定义的变量或常量；对于赋值号左边的 **LVal** 必须是变量。
3. 函数调用形式是 **Ident '(' FuncRParams ')'**，其中的 **FuncRParams** 表示实际参数。实际参数的类型和个数必须与 **Ident** 对应的函数定义的形参完全匹配。
4. **SysY** 中算符的优先级与结合性与 C 语言一致，在上面的 **SysY** 文法中已体现出优先级与结合性的定义。

一元表达式 UnaryExp

1. 相邻两个 **UnaryOp** 不能相同，如 **int a = ++-i;**，但是 **int a = ++-+i;** 是可行的。
2. **UnaryOp** 为 **!** 只能出现在条件表达式中。
3. 显式类型转换 (**BType**) **UnaryExp** 中的 **BType** 只能是基本类型 **int** 或 **char**。

常量表达式 ConstExp

1. **ConstExp** 在 **SysY** 中代表可在编译期求值的表达式，可以是 **int 型表达式** 或 **char 型表达式**。
2. **ConstExp** 在编译期内是可被求值的，且需要符合 C 语言标准中“整型常量表达式 (integer constant expression)”的约束：其操作数只能为整型常量、字符常量，不能访问其它变量、不能进行函数调用（即使访问的变量、进行的函数调用理论上是能在编译期求值的）。
3. **ConstExp** 中不能出现 **int** 和 **char** 类型的混合运算，必须保证类型一致性（如果使用了显式类型转换，则按照转换后的目标类型进行一致性检查）。
4. **ConstExp** 主要用于以下场景：
 - 数组定义时指定各维长度（此时必须是 **int** 型常量表达式）
 - 常量定义时的初始值
 - 全局变量的初始值

5. 常量表达式中允许包含显式类型转换 (BType) UnaryExp，且在编译期对其进行求值（即支持常量折叠，例如将字符常量强转为 `int`，或将整型常量强转为 `char`，转换后的值在编译期直接计算出来）。

六、错误处理

以下内容是为了词法分析及以后的实验作业准备的，在完成文法解读作业时请编写符合上面文法的正确的源程序！

从词法分析作业开始，我们每次作业都要求同学们开发的编译器能够处理正确的程序与错误的程序，对于正确的程序按照每次作业要求输出正确结果，对于错误的程序输出所有的错误的行号和错误类别码。

错误主要分为三类，词法分析部分错误，语法分析部分错误，语义分析部分错误。同学们在涉及编译器时，应该完成这三个部分的所有错误处理之后再进行错误的输出。

后续实验中，对于错误的源程序完成语义分析后不进行中间代码生成。

下面列出的错误并不能覆盖所有源程序中可能出现的错误，但只要求编译器能处理下面列出的错误，同时保证测试用例只涉及下面列出的错误。

错误类型列表

- 错误类别 a 为词法分析中会出现的错误。
- 错误类别 i, j, k 为语法分析中会出现的错误。
- 剩余错误类别均为语义分析中会出现的错误。

错误类型	错误类别码	解释	对应文法及出错符号，...表示省略该条规则其他部分
非法符号	a	出现了 <code>&</code> 和 <code> </code> 这两个符号，应该将其当做 <code>&&</code> 与 <code> </code> 进行处理，但是在记录单词名称的时候仍记录 <code>&</code> 和 <code> </code> ，报错行号为 <code>'&'</code> 或 <code>' '</code> 所在的行号。	<code>LAndExp</code> $\rightarrow$ <code>LAndExp</code> <code>'&&'</code> <code>EqExp</code> <code>/></code> <code>LOrExp</code> $\rightarrow$ <code>LOrExp</code> <code>' '</code> <code>LAndExp</code>
名字重定义	b	函数名或者变量名在 当前作用域 下重复定义。注意，变量一定是同一级作用域下才会判定出错，不同级作用域下，内层会覆盖外层定义。报错行号 <code>Ident</code> 所在行数。	<code>ConstDef</code> $\rightarrow$ <code>Ident</code> ... <code><</code> <code>/></code> <code>VarDef</code> $\rightarrow$... <code>Ident</code> ... <code>Ident</code> ... <code>FuncDef</code> $\rightarrow$ <code>FuncType</code> <code>Ident</code> ... <code><</code> <code>/></code> <code>FuncFParam</code> $\rightarrow$ <code>BType</code> <code>Ident</code> ...
未定义的名字	c	使用了未定义的标识符报错行号为 <code>Ident</code> 所在行数。	<code>LVal</code> $\rightarrow$ <code>Ident</code> ... <code>UnaryExp</code> $\rightarrow$ <code>Ident</code> ...
函数参数个数不匹配	d	函数调用语句中，参数个数与函数定义中的参数个数不匹配。报错行号为函数调用语句的 函数名 所在行数。	<code>UnaryExp</code> $\rightarrow$ <code>Ident</code> <code>'('</code> [<code>FuncRParams</code>] <code>')'</code>
类型不匹配	e	当表达式中出现多类型(指 <code>int/char/int[]/char[]/void</code> 中任意类型)	<code>Stmt</code> $\rightarrow$ <code>LVal</code> <code>'='</code> <code>Exp</code> <code> ';' </code> <code>Stmt</code> $\rightarrow$ <code>'return'</code> [<code>Exp</code>]

错误类型	错误类别码	解释	对应文法及出错符号, ...表示省略该条规则其他部分
		混合运算时, 或者有返回值的函数返回的类型与定义的返回类型不一致, 或者赋值语句中左值和右值类型不一致, 或者函数参数传递时实参类型与形参类型不一致, 或者 <code>switch</code> 语句中 <code>case</code> 的 <code>Number</code> 类型与 <code>switch</code> 表达式类型不一致, 或者 <code>printf</code> 语句中格式字符串与表达式类型不一致时, 或者函数标识符与变量相互误用时, 报此错误。报错行号为 发生类型不匹配的位置 所在行号。	<pre> ';' UnaryExp → Ident '(' [FuncRParams] ')' CaseStmt → 'case' Number ':' ... Stmt → 'printf' '(' StringConst {' , ' Exp } ')' ';' </pre>
无返回值的函数存在不匹配的 <code>return</code> 语句	f	报错行号为 <code>'return'</code> 所在行号。	<pre> Stmt → 'return' { '[' Exp ']' } ';' </pre>
有返回值的函数缺少 <code>return</code> 语句	g	只需要考虑函数末尾是否存在 <code>return</code> 语句, 无需考虑控制流 。报错行号为函数 结尾的 <code>'</code> 所在行号。	<pre> FuncDef → FuncType Ident '(' [FuncFParams] ')' Block MainFuncDef → 'int' 'main' '(' ')' Block </pre>
不能改变常量的值	h	LVal 为常量时, 不能对其修改。报错行号为 LVal 所在行号。	<pre> Stmt → LVal '=' Exp ';' </pre>
缺少分号	i	报错行号为分号 前一个非终结符 所在行号。	<code>Stmt</code> , <code>ConstDecl</code> 及 <code>VarDecl</code> 中的 <code>;</code>
缺少右小括号 <code>)</code>	j	报错行号为右小括号 前一个非终结符 所在行号。	函数调用 (<code>UnaryExp</code>)、函数定义 (<code>FuncDef</code> , <code>MainFuncDef</code>)、 <code>Stmt</code> 、 <code>PrimaryExp</code> 及 显式类型转换 <code>'(' BType ')'</code> <code>UnaryExp</code> 中的 <code>)</code>
缺少右中括号 <code>]</code>	k	报错行号为右中括号 前一个非终结符 所在行号。	数组定义 (<code>ConstDef</code> , <code>VarDef</code> , <code>FuncFParam</code>) 和使用 (<code>LVal</code>) 中的 <code>]</code>
<code>printf</code> 中格式字符与表达式个数不匹配	l	报错行号为 <code>'printf'</code> 所在行号。	<pre> 'printf' '(' StringConst {' , ' Exp } ')' ';' </pre>
在非循环块中使用	m	报错行号为 <code>'break'</code> 与 <code>'continue'</code> 所在行号。 <code>break</code> 可以出现在 <code>while</code> 循环或 <code>switch</code> 语句	<pre> Stmt → 'break' ';' 'continue' ';' </pre>

错误类型	错误类别码	解释	对应文法及出错符号，...表示省略该条规则其他部分
break 和 continue 语句		中，continue 只能出现在 while 循环中。	
switch 语句中 case 和 default 重复	n	switch 语句中出现了重复的 case 常量值，或者出现了多个 default 分支。报错行号为重复的 'case' 或 'default' 所在行号。	CaseStmt → 'case' Number ':' { Stmt } 'default' ':' { Stmt }

错误输出示例

输出结构

错误的行号 错误的类别码（中间仅用一个空格间隔）

样例输入

```
const int const1 = 1, const2 = -100;
int change1;
int gets1(int var1,int var2){
    const1 = 999;
    change1 = var1 + var2          return (change1);
}
int main(){
    change1 = 10;
    printf("Hello World");
    return 0;
}
```

样例输出

4 h
5 i

文法符号与错误类型对应

为方便对照, 下文给出了文法符号与 可能存在 的错误的对应关系:

编译单元 $\text{CompUnit} \rightarrow \{\text{Decl}\} \{\text{FuncDef}\} \text{MainFuncDef}$

声明 $\text{Decl} \rightarrow \text{ConstDecl} \mid \text{VarDecl}$

常量声明 $\text{ConstDecl} \rightarrow \text{'const'} \text{BType} \text{ConstDef} \{ \text{' ,' ConstDef} \} \text{' ;'}$ // i

基本类型 $\text{BType} \rightarrow \text{'int'} \mid \text{'char'}$

常量定义 $\text{ConstDef} \rightarrow \text{Ident} [\text{' [' ConstExp ']}] \text{'=' ConstInitVal}$ // b, k

常量初值 $\text{ConstInitVal} \rightarrow \text{ConstExp} \mid \text{'{' [ConstExp \{ \text{' ,' ConstExp} \}] \text{'}'}$

变量声明 $\text{VarDecl} \rightarrow [\text{'static'}] \text{BType} \text{VarDef} \{ \text{' ,' VarDef} \} \text{' ;'}$ // i

变量定义 $\text{VarDef} \rightarrow \text{Ident} [\text{' [' ConstExp ']}] \mid \text{Ident} [\text{' [' ConstExp ']}] \text{'=' InitVal}$ // b, k

变量初值 $\text{InitVal} \rightarrow \text{Exp} \mid \text{'{' [Exp \{ \text{' ,' Exp} \}] \text{'}'}$

函数定义 $\text{FuncDef} \rightarrow \text{FuncType} \text{Ident} \text{'(' [FuncFParams] \text{'})' Block}$ // b, e, g, j

主函数定义 $\text{MainFuncDef} \rightarrow \text{'int'} \text{'main'} \text{'(' ')} \text{' Block}$ // g, j

函数类型 $\text{FuncType} \rightarrow \text{'void'} \mid \text{'int'} \mid \text{'char'}$

函数形参表 $\text{FuncFParams} \rightarrow \text{FuncFParam} \{ \text{' ,' FuncFParam} \}$

函数形参 $\text{FuncFParam} \rightarrow \text{BType} \text{Ident} [\text{' [' ']}]$ // b, k

语句块 $\text{Block} \rightarrow \text{'{' \{ BlockItem \} \text{'}'}$

语句块项 $\text{BlockItem} \rightarrow \text{Decl} \mid \text{Stmt}$

语句 $\text{Stmt} \rightarrow \text{LVal} \text{'=' Exp} \text{' ;'}$ // e, h, i
 $\mid [\text{Exp}] \text{' ;'}$ // i
 $\mid \text{Block}$
 $\mid \text{'if'} \text{'(' Cond \text{'})' Stmt} [\text{'else'} \text{Stmt}]$ // j
 $\mid \text{'while'} \text{'(' Cond \text{'})' Stmt}$ // j
 $\mid \text{'switch'} \text{'(' Exp \text{'})' \text{'{' \{ CaseStmt \} \text{'}'}}$ // j
 $\mid \text{'break'} \text{' ;'}$ $\mid \text{'continue'} \text{' ;'}$ // i, m
 $\mid \text{'return'} [\text{Exp}] \text{' ;'}$ // e, f, i
 $\mid \text{'printf'} \text{'(' StringConst \{ \text{' ,' Exp} \} \text{'})' \text{' ;'}}$ // e, l

Case语句 $\text{CaseStmt} \rightarrow \text{'case'} \text{Number} \text{' :' \{ Stmt \} } \mid \text{'default'} \text{' :' \{ Stmt \} }$ // e, n

表达式 $\text{Exp} \rightarrow \text{AddExp}$

条件表达式 $\text{Cond} \rightarrow \text{LOrExp}$

左值表达式 $\text{LVal} \rightarrow \text{Ident} [\text{' [' Exp ']}]$ // c, k

基本表达式 $\text{PrimaryExp} \rightarrow '(' \text{Exp} ')' \mid \text{LVal} \mid \text{Number} \mid j$

数值 $\text{Number} \rightarrow \text{IntConst} \mid \text{CharConst}$

一元表达式 $\text{UnaryExp} \rightarrow \text{PrimaryExp} \mid \text{Ident} '(' [\text{FuncRParams}] ')' \mid \text{UnaryOp} \text{UnaryExp} \mid '(' \text{BType} ')' \text{UnaryExp} \mid c, d, e, j$

单目运算符 $\text{UnaryOp} \rightarrow '+' \mid '-' \mid '!' \mid \text{注: '!' 仅出现在条件表达式中}$

函数实参表 $\text{FuncRParams} \rightarrow \text{Exp} \{ ',' \text{Exp} \}$

乘除模表达式 $\text{MulExp} \rightarrow \text{UnaryExp} \mid \text{MulExp} ('*' \mid '/' \mid '\%') \text{UnaryExp}$

加减表达式 $\text{AddExp} \rightarrow \text{MulExp} \mid \text{AddExp} ('+' \mid '-') \text{MulExp}$

关系表达式 $\text{RelExp} \rightarrow \text{AddExp} \mid \text{RelExp} ('<' \mid '>' \mid '<=' \mid '>=') \text{AddExp}$

相等性表达式 $\text{EqExp} \rightarrow \text{RelExp} \mid \text{EqExp} ('==' \mid '!=') \text{RelExp}$

逻辑与表达式 $\text{LAndExp} \rightarrow \text{EqExp} \mid \text{LAndExp} '&&' \text{EqExp} \mid a$

逻辑或表达式 $\text{LOrExp} \rightarrow \text{LAndExp} \mid \text{LOrExp} '||' \text{LAndExp} \mid a$

常量表达式 $\text{ConstExp} \rightarrow \text{AddExp} \mid \text{注: 使用的 Ident 必须是常量}$

特别说明

1. 错误 i, j, k 类型中的“前一个非终结符”强调的是在文法规则里出现在 `;` `)` `]` 之前的非终结符号，在分析中处理的是该非终结符产生的终结符号串的最后一个符号，也就是 `;` `)` `]` 本应该正常出现的位置的上一个单词。
2. 所有错误都不会出现恶意换行的情况，包括字符、字符串中的换行符、函数调用等等。所谓恶意换行是指会使得错误处理难以正确进行的换行。
3. 其他类型的错误，错误的行号以能够断定发现出错的第一个符号的行号为准。例如有返回值的函数缺少返回语句的错误，只有当识别到函数末尾的 `}` 时仍未出现返回语句，才可以断定出错，报错行号即为 `}` 的行号。
4. 为了避免因为测试程序中的错误导致出现语法二义性，使得语法树以错误的方式建立，我们保证，所有会导致语法树不能正确建立的错误都不会出现。
5. 每一行中最多只有一个错误。
6. 同一作用域下函数和变量之间不能有相同的名字。
7. 对于一个名字重定义的函数，也应该完整分析函数内部是否具有其它错误。
8. 对于调用函数时，参数类型不匹配一共有以下几种情况的不匹配：
 - 传递数组给变量。
 - 传递变量给数组。
 - 传递 `int` 类型给 `char` 类型参数。

- 传递 `char` 类型给 `int` 类型参数。
9. 对于 `e` 类型错误，包含所有类型不匹配的错误，本文法共5种类型，分别是 `int`、`char`、`int[]`、`char[]`、`void`。对于不符合下述类型匹配规则的错误，均应当返回 `e` 类型错误：
- 不同类型不能进行任何混合运算，包括赋值、算数运算、关系运算。
 - 函数调用时，实参类型必须与形参类型完全匹配。
 - 函数返回值类型必须与函数定义时声明的返回值类型完全匹配。
 - `switch` 语句中，`case` 标签的常量表达式类型必须与 `switch` 表达式的类型完全匹配，且必须是 `int` 或 `char`。
 - 数组下标必须是 `int` 类型。
 - 不能将非数组类型变量进行数组访问。
10. 对于显式类型转换 (`BType`) `UnaryExp`，测试用例保证其操作数 `UnaryExp` 必须是基本类型 (`int` 或 `char`)，且目标类型 `BType` 也是基本类型 (`int` 或 `char`)，不测试对数组或 `void` 等非基本类型的显式转换。如果显式类型转换缺少右小括号 `)`，则应报告 `j` 类型错误，其报错行号为前一个非终结符 `BType` (即关键字 `int` 或 `char`) 所在的行号。
11. 测试程序保证不出现未提到的错误形式